

## What is Abstraction?

**Abstraction** means hiding unnecessary internal details and showing only the essential features to the user.

In simple terms:

You don't need to know *how* something works internally, only *how to use it*.

---

## Real-Life Example

When you drive a car:

- You know how to start, accelerate, and brake.
- You do not need to know how the engine or gearbox works internally.

This is **abstraction** — you use the car's interface without worrying about its internal mechanics.

---

## Abstraction in Python

In Python, abstraction is implemented using **abstract classes and abstract methods**.

This is done through the built-in **abc (Abstract Base Class)** module.

---

## Abstract Class in Python

An **abstract class** is a class that:

- Cannot be instantiated (you cannot create its object directly).
  - Can contain abstract methods (methods declared but not implemented).
  - Is meant to be inherited by other classes which will implement those methods.
- 

## Syntax

```
from abc import ABC, abstractmethod
```

```
class ClassName(ABC):
```

```
    @abstractmethod
```

```
    def method_name(self):
```

```
        pass
```

Any class that inherits from this abstract class must implement the abstract methods.

---

### Example 1: Basic Abstraction Example

```
from abc import ABC, abstractmethod
```

```
class Vehicle(ABC):
```

```
    @abstractmethod
```

```
    def start(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def stop(self):
```

```
        pass
```

```
class Car(Vehicle):
```

```
    def start(self):
```

```
        print("Car has started")
```

```
    def stop(self):
```

```
        print("Car has stopped")
```

```
# v = Vehicle() # Error: Can't create an object of abstract class
```

```
car1 = Car()    # Valid
```

```
car1.start()
```

```
car1.stop()
```

#### Output:

```
Car has started
```

```
Car has stopped
```

#### Explanation:

1. Vehicle is an abstract class.
2. It defines abstract methods start() and stop().

3. Car inherits from Vehicle and provides implementation for those methods.
  4. If Car didn't implement them, Python would raise an error.
- 

### Example 2: Multiple Child Classes

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
```

```
    @abstractmethod
```

```
    def area(self):
```

```
        pass
```

```
class Circle(Shape):
```

```
    def __init__(self, radius):
```

```
        self.radius = radius
```

```
    def area(self):
```

```
        return 3.14 * self.radius * self.radius
```

```
class Rectangle(Shape):
```

```
    def __init__(self, length, breadth):
```

```
        self.length = length
```

```
        self.breadth = breadth
```

```
    def area(self):
```

```
        return self.length * self.breadth
```

```
shapes = [Circle(5), Rectangle(4, 6)]
```

for s in shapes:

```
    print("Area:", s.area())
```

### Output:

Area: 78.5

Area: 24

### Explanation:

Shape defines a common structure (area()), and each subclass provides its own implementation. This shows abstraction because we use the area() method without worrying about how each shape calculates it.

---

### Why Use Abstraction?

| Benefit                   | Description                                                   |
|---------------------------|---------------------------------------------------------------|
| Hides complexity          | Internal working is hidden from the user.                     |
| Promotes code reusability | Common structures can be reused by subclasses.                |
| Improves flexibility      | Changes in implementation do not affect the rest of the code. |
| Maintains clarity         | Developers must follow a consistent design structure.         |

---

### Important Rules of Abstraction

1. You cannot create objects of an abstract class.
  2. All abstract methods must be defined in the child class.
  3. Abstract classes can contain both abstract and normal methods.
  4. Abstract classes act as blueprints for derived classes.
- 

### Example 3: Abstract Class with Normal Method

```
from abc import ABC, abstractmethod
```

```
class Payment(ABC):
```

```
    def show_invoice(self):
```

```
        print("Invoice generated successfully.")
```

```
    @abstractmethod
```

```
    def pay(self):
```

pass

```
class CreditCardPayment(Payment):  
    def pay(self):  
        print("Payment done using Credit Card.")
```

```
p = CreditCardPayment()  
p.show_invoice()  
p.pay()
```

**Output:**

Invoice generated successfully.

Payment done using Credit Card.

**Explanation:**

Here, Payment has both an abstract method (pay) and a normal method (show\_invoice). The child class CreditCardPayment provides the implementation for pay().

---

**Summary**

| Concept        | Description                                                    |
|----------------|----------------------------------------------------------------|
| Abstraction    | Hiding internal details and showing only necessary information |
| Implementation | Using abstract classes and abstract methods                    |
| Module Used    | abc                                                            |
| Purpose        | Provide a common interface or structure for derived classes    |

---

**Key Takeaway**

- Abstraction hides implementation details and exposes only the required interface.
- It helps design clean, structured, and maintainable code.
- Use abc and @abstractmethod to define abstract classes and enforce method implementation.